

**МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ**

**Ордена трудового Красного Знамени федеральное государственное
бюджетное**

**образовательное учреждение высшего образования
«Московский технический университет связи и информатики»**

Кафедра Программная инженерия

Отчет по лабораторной работе 2
Деревья и быстрые запросы к данным

Выполнил: студент группы БПИ2403

Рыбаченок Вадим Александрович
Руководитель: Борзов Вадим Максимович

Москва, 2026

Цель работы

Изучить применение древовидных структур для поиска и обработки запросов. Реализовать операции BST, исследовать зависимость поиска от высоты и показать повороты AVL. Освоить префиксные суммы, дерево Фенвика и дерево отрезков для диапазонных запросов и точечных обновлений.

В этой работе я изучаю деревья для поиска по ключу и структуры для запросов к части массива. Сначала строю бинарное дерево поиска, проверяю вставку, поиск, обходы и удаление. Затем сравниваю обычный подсчёт суммы с префиксными суммами, деревом Фенвика и деревом отрезков. В самостоятельном задании применяю эти структуры к измерениям датчика.

Я выполняю все 13 заданий в одном проекте `tree_range_lab`. Использую только стандартную библиотеку Python. В `bst.py` находятся общие операции с деревом, в `ranges.py` — операции с диапазонами, а в `main.py` — примеры заданий и их запуск. Общие функции повторно используются, поэтому я привожу их полный код с комментариями один раз и дальше ссылаюсь на соответствующее задание.

Структура проекта и запуск

```
tree_range_lab/  
  bst.py  
  ranges.py  
  main.py  
  README.md
```

```
Все задания:    python main.py  
Одно задание:  python main.py 13
```

Я запускаю команды из папки `tree_range_lab`. Если команда `python` не найдена, можно использовать `py main.py`. Дополнительные пакеты не нужны. Подробная инструкция сохранена рядом с отчётом в `Инструкция.md`; полный фактический вывод находится в `Результаты.txt`.

Основные обозначения

Во всех практических запросах `left` и `right` — индексы с нуля, обе границы включаются. Например, запрос `[2, 6]` обращается к пяти элементам с индексами 2, 3, 4, 5 и 6. В срезе Python правая граница не включается, поэтому для проверки я использую `data[left:right + 1]`. Внутри Фенвика индексы начинаются с 1, но его методы тоже принимают внешние индексы с нуля.

Высота дерева считается по числу рёбер на самом длинном пути от корня до листа. У одного листа высота 0, у пустого дерева я принимаю высоту -1. Это позволяет использовать простую рекурсивную формулу. Для повторного ключа правило отдельное: `insert` оставляет дерево без изменений. Все рабочие идентификаторы датчиков уникальны.

Узел и ссылки

Узел — отдельный объект `Node`. Поле `key` хранит его значение. Поля `left` и `right` хранят ссылки на другие узлы, а не их копии. Если потомка нет, в поле находится `None`. Запись `root.left.key` означает: взять левого потомка корня и прочитать его ключ.

self и создание объекта

`self` — ссылка на тот объект, чей метод сейчас выполняется. Вызов `Node(8)` создаёт объект и вызывает его метод `__init__`. Внутри этого метода `self.key = key` сохраняет число 8 именно в созданном узле. У разных объектов свои поля, даже если названия полей одинаковые.

Рекурсия и return

Рекурсия — вызов функцией самой себя для меньшей части задачи. Например, `insert` переходит к левому или правому поддереву. Пустая ветвь является условием остановки. `return` возвращает результат и сразу завершает текущий вызов. Поэтому возвращённый корень нужно записать обратно в `root`, `root.left` или `root.right`.

Списки и пары

Квадратные скобки создают список. Запись `data[i]` получает один его элемент. В паре `(left, right)` сохраняются две границы запроса. Строка `left, right = queries[i]` раскладывает эту пару в две переменные. Такой же приём используется для пары (узел, число посещений) и тройки (сумма, минимум, максимум).

Циклы и диапазоны

`range(n)` задаёт числа от 0 до $n - 1$. Поэтому `range(1, 14)` даёт номера заданий 1–13, а `range(left, right + 1)` включает правый индекс запроса. Цикл `while` продолжается, пока его условие истинно. Оператор `+=` прибавляет значение к переменной, `//` выполняет целочисленное деление, `%` получает остаток от деления.

Подключения и общие исходные данные `main.py`

```
# Подключаю стандартный модуль для воспроизводимых случайных данных.
import random
# Подключаю модуль для чтения аргументов командной строки.
import sys
# Подключаю часы perf_counter для измерения времени выполнения.
import time

# Подключаю перечисленные функции или классы из модуля bst; определения ниже разбирать
# заново не нужно.
from bst import Node, insert, build_bst, search, preorder, inorder, postorder
# Подключаю перечисленные функции или классы из модуля bst; определения ниже разбирать
# заново не нужно.
from bst import delete, height, balance_factor, rotate_right, rotate_left, show_tree
# Подключаю перечисленные функции или классы из модуля ranges; определения ниже разбирать
# заново не нужно.
from ranges import range_sum_naive, build_prefix, range_sum_prefix
# Подключаю перечисленные функции или классы из модуля ranges; определения ниже разбирать
# заново не нужно.
from ranges import FenwickTree, SegmentTree

# Задаю ключи и их порядок вставки из методички; этот список переиспользуется.
KEYS = [8, 4, 12, 2, 6, 10, 14]
# Задаю небольшой массив из лекции для ручной проверки сумм.
EXAMPLE = [5, 2, 7, 3, 6, 1, 4, 8]
# Задаю шесть диапазонов: один элемент, весь массив и промежуточные части.
CHECK_RANGES = [(0, 0), (0, 7), (2, 6), (1, 5), (7, 7), (3, 4)]
```

Выбор задания в `main.py`

Список `tasks` хранит функции, а не их результаты. У функции без скобок нет немедленного запуска. В строке `tasks[number - 1]()` сначала выбирается функция по индексу, затем скобки вызывают её. Номера заданий начинаются с 1, поэтому индекс списка равен номеру минус 1.

```

# Запускаю примеры только при прямом запуске main.py, а не при его импорте.
if __name__ == "__main__":
    # Создаю список самих функций; без скобок они пока не выполняются.
    tasks = [task1, task2, task3, task4, task5, task6, task7,
              # Дополняю список функциями заданий 8..13 в порядке их номеров.
              task8, task9, task10, task11, task12, task13]
    # Допускаю только имя файла и не более одного аргумента с номером.
    if len(sys.argv) > 2:
        # Завершаю запуск с подсказкой, если передано слишком много аргументов.
        raise SystemExit("Запуск: python main.py [номер задания 1..13]")
    # Если номер задания передан, выбираю запуск только этого задания.
    if len(sys.argv) == 2:
        # Проверяю, что аргумент состоит из цифр и задаёт номер от 1 до 13.
        if not sys.argv[1].isdigit() or not 1 <= int(sys.argv[1]) <= 13:
            # Показываю понятное сообщение вместо попытки вызвать несуществующее задание.
            raise SystemExit("Укажите номер задания от 1 до 13")
        # Преобразую текстовый аргумент в число и помещаю его в список из одного номера.
        numbers = [int(sys.argv[1])]
    # Если предыдущие условия не выполнены, выполняю эту оставшуюся ветвь.
    else:
        # Без аргумента выбираю все номера 1..13; число 14 не включается.
        numbers = range(1, 14)
    # По очереди запускаю выбранные номера заданий.
    for number in numbers:
        # Вывожу «=== Задание ===» и соответствующие значения.
        print("\n=== Задание", number, "===")
        # Беру функцию по индексу number - 1 и вызываю её с помощью скобок ().
        tasks[number - 1]()

```

Задание 1 Создание бинарного дерева

Условие задания

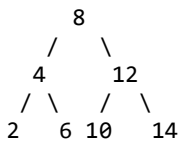
1. Создайте класс Node с атрибутами key, left и right.
2. Вручную создайте бинарное дерево с корнем 8 и узлами 4, 12, 2, 6, 10 и 14.
3. Выведите значения корня, его потомков и одного из листьев.

Ход решения

Я создаю семь отдельных объектов Node и вручную связываю их через left и right. Корнем становится 8, его потомками — 4 и 12. Узлы 2, 6, 10 и 14 не имеют потомков и являются листьями. Здесь вставка ещё не используется: связи задаются непосредственно в коде.

Дополнительная функция show_tree нужна только для вывода структуры. Каждые два пробела обозначают следующий уровень. Л — левый потомок, П — правый. Эта функция не сортирует ключи и не меняет дерево.

Схема



Код bst.py Node

```
# Объявляю класс узла дерева: он хранит ключ и ссылки на потомков.
class Node:
    # Создаю объект и заполняю его начальные поля; self означает этот объект.
    def __init__(self, key):
        # Сохраняю переданный ключ в поле key данного узла.
        self.key = key
        # Левого потомка пока нет: None обозначает отсутствие узла.
        self.left = None
        # Правого потомка пока нет: None обозначает отсутствие узла.
        self.right = None
```

Код bst.py show_tree

```
# Объявляю вывод дерева: level задаёт отступ, side подписывает ветвь.
def show_tree(root, level=0, side="Корень"):
    # Проверяю, пусто ли поддерево; в этом случае спуск продолжать нельзя.
    if root is None:
        # Немедленно завершаю функцию; ниже в этой ветви делать нечего.
        return
    # Вывожу полученный результат текущего этапа.
    print(" " * level + side + ": " + str(root.key))
    # Печатаю левую ветвь на следующем уровне с подписью Л.
    show_tree(root.left, level + 1, "Л")
```

```
# Печатаю правую ветвь на следующем уровне с подписью П.  
show_tree(root.right, level + 1, "П")
```

Код main.py task1

```
# Объявляю демонстрацию задания 1; она выполняется при вызове.  
def task1():  
    # Вручную создаю корневой узел с ключом 8.  
    root = Node(8)  
    # Связываю корень с левым потомком 4.  
    root.left = Node(4)  
    # Связываю корень с правым потомком 12.  
    root.right = Node(12)  
    # Создаю узел 2 как левого потомка узла 4.  
    root.left.left = Node(2)  
    # Создаю узел 6 как правого потомка узла 4.  
    root.left.right = Node(6)  
    # Создаю узел 10 как левого потомка узла 12.  
    root.right.left = Node(10)  
    # Создаю узел 14 как правого потомка узла 12.  
    root.right.right = Node(14)  
    # Вывожу «Корень:» и соответствующие значения.  
    print("Корень:", root.key)  
    # Вывожу «Потомки корня:» и соответствующие значения.  
    print("Потомки корня:", root.left.key, root.right.key)  
    # Вывожу «Один из листьев:» и соответствующие значения.  
    print("Один из листьев:", root.left.left.key)  
    # Вызываю вывод структуры дерева; значения и связи при этом не меняются.  
    show_tree(root)
```

Результат выполнения

```
Корень: 8  
Потомки корня: 4 12  
Один из листьев: 2  
Корень: 8  
  Л: 4  
    Л: 2  
    П: 6  
  П: 12  
    Л: 10  
    П: 14
```

Объяснение результата и вывод

Программа вывела корень 8, потомков 4 и 12 и лист 2. Схема подтверждает, что у каждого узла не больше двух потомков. Например, `root.left.left.key` читает ключ узла 2: от корня я дважды перехожу по левой ссылке.

Вывод: бинарное дерево можно представить отдельными объектами, связанными ссылками. Само наличие двух ссылок ещё не задаёт правило поиска; для этого дальше использую свойство BST.

Задание 2 Вставка в бинарное дерево поиска

Условие задания

1. Реализуйте функцию `insert(root, key)`, сохраняющую свойство BST.
2. Постройте дерево последовательной вставкой значений 8, 4, 12, 2, 6, 10, 14.
4. Не используйте встроенные структуры, автоматически поддерживающие сортировку элементов.

Ход решения

Я начинаю с `root = None` и по очереди передаю в `insert` числа 8, 4, 12, 2, 6, 10, 14. Если текущая ветвь пустая, создаётся новый `Node`. Меньший ключ направляется влево, больший — вправо. После рекурсивного вызова я сохраняю ссылку на корень изменённой ветви.

Если ключ равен текущему, оба сравнения `<` и `>` не выполняются. Функция возвращает прежний узел, поэтому повторный ключ не добавляется. Встроенные структуры, автоматически поддерживающие сортировку, при построении не используются.

Код `bst.py insert`

```
# Объявляю вставку ключа; функция возвращает корень полученного поддерева.
def insert(root, key):
    # Проверяю, пусто ли поддерево; в этом случае спуск продолжать нельзя.
    if root is None:
        # Создаю новый узел в пустой позиции и возвращаю его вызывающему коду.
        return Node(key)
    # Если ключ меньше текущего, выбираю левое поддерево.
    if key < root.key:
        # Вставляю ключ слева и сохраняю возвращённый корень левого поддерева.
        root.left = insert(root.left, key)
    # Если ключ больше текущего, выбираю правое поддерево.
    elif key > root.key:
        # Вставляю ключ справа и сохраняю возвращённый корень правого поддерева.
        root.right = insert(root.right, key)
    # Возвращаю корень поддерева; вызывающий код сохраняет эту ссылку.
    return root
```

Код `bst.py build_bst`

```
# Объявляю построение BST последовательной вставкой ключей из values.
def build_bst(values):
    # Начинаю с пустого дерева; первая вставка создаст его корень.
    root = None
    # Перебираю ключи в заданном порядке; порядок влияет на форму дерева.
    for key in values:
        # Вставляю очередной ключ и сохраняю корень после вставки.
        root = insert(root, key)
    # Возвращаю корень поддерева; вызывающий код сохраняет эту ссылку.
    return root
```

Код main.py task2

```
# Объявляю демонстрацию задания 2; она выполняется при вызове.
def task2():
    # Начинаю с пустого дерева; первая вставка создаст его корень.
    root = None
    # Перебираю ключи из общего списка в первоначальном порядке.
    for key in KEYS:
        # Вставляю очередной ключ и сохраняю корень после вставки.
        root = insert(root, key)
    # Вывожу «Порядок вставки:» и соответствующие значения.
    print("Порядок вставки:", KEYS)
    # Вызываю вывод структуры дерева; значения и связи при этом не меняются.
    show_tree(root)
```

Результат выполнения

```
Порядок вставки: [8, 4, 12, 2, 6, 10, 14]
Корень: 8
  Л: 4
    Л: 2
    П: 6
  П: 12
    Л: 10
    П: 14
```

Объяснение результата и вывод

Форма дерева совпала с ручной схемой задания 1. Ключи 2 и 6 расположены в левом поддереве 8, а 10 и 14 — в правом. Для узла 4 число 2 меньше, а 6 больше; правило проверяется не только у корня.

Вывод: `insert` сохраняет правило «все ключи слева меньше, все справа больше». Функция `build_bst` делает те же вставки в цикле и нужна для повторного построения дерева в следующих заданиях. Сложность одной вставки $O(h)$, где h — высота дерева.

Задание 3 Поиск элемента

Условие задания

1. Реализуйте функцию `search(root, key)`.
2. Дополните функцию подсчётом количества посещённых узлов.
3. Выполните поиск значения, расположенного близко к корню, значения из нижнего уровня и отсутствующего значения.

Ход решения

Я строю исходное BST и ищу 4, 14 и 5. Переменная `current` сначала указывает на корень. При каждом посещении настоящего узла увеличиваю `visited`. Переход к `None` не считается посещением узла, потому что объекта в этой позиции нет.

Функция возвращает два результата: найденный объект или `None` и число посещений. Запись `node, visited = search(root, key)` распределяет их по переменным. Проверка `node is not None` даёт `True`, если объект найден, и `False`, если его нет.

Код `bst.py search`

```
# Объявляю поиск: возвращаю найденный узел и число посещённых узлов.
def search(root, key):
    # Начинаю поиск с корня; current хранит текущий посещаемый узел.
    current = root
    # До начала поиска число посещённых узлов равно нулю.
    visited = 0
    # Продолжаю поиск, пока текущий узел существует.
    while current is not None:
        # Увеличиваю счётчик на один при посещении очередного узла.
        visited += 1
        # Проверяю, совпадает ли искомый ключ с ключом текущего узла.
        if key == current.key:
            # Возвращаю найденный узел и счётчик; поиск сразу завершается.
            return current, visited
        # Меньший искомый ключ может находиться только в левом поддереве.
        if key < current.key:
            # Перехожу по ссылке на левого потомка.
            current = current.left
        # Если предыдущие условия не выполнены, выполняю эту оставшуюся ветвь.
        else:
            # Перехожу по ссылке на правого потомка.
            current = current.right
    # Ключ не найден; возвращаю None и число уже просмотренных узлов.
    return None, visited
```

Код `main.py task3`

```
# Объявляю демонстрацию задания 3; она выполняется при вызове.
def task3():
    # Строю новое BST из семи ключей; предыдущие задания его не изменяют.
    root = build_bst(KEYS)
```

```
# Проверяю ключ около корня, ключ нижнего уровня и отсутствующий ключ.  
for key in [4, 14, 5]:  
    # Разбираю пару ответа поиска на найденный узел и число посещений.  
    node, visited = search(root, key)  
    # Вывожу «Ключ: Найден: Посещено:» и соответствующие значения.  
    print("Ключ:", key, "Найден:", node is not None, "Посещено:", visited)
```

Результат выполнения

```
Ключ: 4 Найден: True Посещено: 2  
Ключ: 14 Найден: True Посещено: 3  
Ключ: 5 Найден: False Посещено: 3
```

Объяснение результата и вывод

Для ключа 4 путь равен $8 \rightarrow 4$, поэтому посещены два узла. Для ключа 14 путь $8 \rightarrow 12 \rightarrow 14$ содержит три узла. Для отсутствующего 5 поиск проходит $8 \rightarrow 4 \rightarrow 6$, затем переходит в пустую левую ветвь узла 6 и завершается с None. Посещены также три настоящих узла.

Вывод: поиск идёт только по одной ветви, выбранной сравнениями. Его сложность $O(h)$. На этом небольшом дереве отсутствие ключа не требует обходить все семь узлов.

Задание 4 Обходы дерева

Условие задания

1. Реализуйте функции `preorder(root)`, `inorder(root)` и `postorder(root)`.
2. Выполните все три обхода для построенного BST.
3. Объясните, почему `inorder`-обход BST формирует отсортированную последовательность ключей.

Ход решения

Я использую три рекурсивные функции. Они отличаются только тем, в какой момент добавляют текущий `key` в `result`: перед обеими ветвями, между ними или после них. При первом вызове создаётся новый список, а рекурсивные вызовы используют тот же список.

В параметре `result=None` нельзя подменять `None` общим пустым списком в объявлении функции. Новый список создаётся внутри вызова, поэтому результаты разных обходов не смешиваются. Добавление `append` выполняется на месте, без копирования всего результата при каждом узле.

Схема

Preorder: узел → левая ветвь → правая ветвь
Inorder: левая ветвь → узел → правая ветвь
Postorder: левая ветвь → правая ветвь → узел

Код `bst.py preorder`

```
# Объявляю обход: узел, левое поддерево, правое поддерево.
def preorder(root, result=None):
    # При первом вызове список результата не передан; создаю новый список.
    if result is None:
        # Создаю пустой список для накопления результата.
        result = []
    # Обрабатываю только существующий узел; пустая ветвь ничего не добавляет.
    if root is not None:
        # Добавляю ключ текущего узла в общий список обхода.
        result.append(root.key)
        # Рекурсивно обхожу левую ветвь, используя тот же список результата.
        preorder(root.left, result)
        # Рекурсивно обхожу правую ветвь, используя тот же список результата.
        preorder(root.right, result)
    # Возвращаю накопленный результат вызывающему коду.
    return result
```

Код `bst.py inorder`

```
# Объявляю обход: левое поддерево, узел, правое поддерево.
def inorder(root, result=None):
```

```

# При первом вызове список результата не передан; создаю новый список.
if result is None:
    # Создаю пустой список для накопления результата.
    result = []
# Обрабатываю только существующий узел; пустая ветвь ничего не добавляет.
if root is not None:
    # Сначала обхожу левую ветвь; её ключи меньше текущего.
    inorder(root.left, result)
    # Добавляю ключ текущего узла в общий список обхода.
    result.append(root.key)
    # Обхожу правую ветвь; её ключи больше текущего.
    inorder(root.right, result)
# Возвращаю накопленный результат вызывающему коду.
return result

```

Код bst.py postorder

```

# Объявляю обход: левое поддерево, правое поддерево, узел.
def postorder(root, result=None):
    # При первом вызове список результата не передан; создаю новый список.
    if result is None:
        # Создаю пустой список для накопления результата.
        result = []
    # Обрабатываю только существующий узел; пустая ветвь ничего не добавляет.
    if root is not None:
        # Обхожу левых потомков раньше текущего узла.
        postorder(root.left, result)
        # Обхожу правых потомков раньше текущего узла.
        postorder(root.right, result)
        # Добавляю ключ текущего узла в общий список обхода.
        result.append(root.key)
    # Возвращаю накопленный результат вызывающему коду.
    return result

```

Код main.py task4

```

# Объявляю демонстрацию задания 4; она выполняется при вызове.
def task4():
    # Строю новое BST из семи ключей; предыдущие задания его не изменяют.
    root = build_bst(KEYS)
    # Вывожу «Preorder:» и соответствующие значения.
    print("Preorder:", preorder(root))
    # Вывожу «Inorder:» и соответствующие значения.
    print("Inorder:", inorder(root))
    # Вывожу «Postorder:» и соответствующие значения.
    print("Postorder:", postorder(root))

```

Результат выполнения

Preorder: [8, 4, 2, 6, 12, 10, 14]
Inorder: [2, 4, 6, 8, 10, 12, 14]
Postorder: [2, 6, 4, 10, 14, 12, 8]

Объяснение результата и вывод

Preorder начинается с корня 8. Inorder начинается с самого левого ключа 2 и получает возрастающую последовательность. Postorder заканчивается корнем 8, потому что сначала обрабатывает обоих его потомков.

Inorder сортирует ключи именно благодаря свойству BST: вся левая ветвь меньше текущего ключа, а правая больше. В произвольном бинарном дереве такой порядок посещения не гарантирует сортировку. Каждый узел посещается один раз: время $O(n)$, список результата занимает $O(n)$, стек рекурсии — $O(h)$.

Задание 5 Удаление из BST

Условие задания

1. Реализуйте функцию `delete(root, key)`.
2. Продемонстрируйте удаление листового узла.
3. Подготовьте дерево так, чтобы можно было удалить узел с одним потомком, и выполните удаление.
4. Удалите узел с двумя потомками.

Ход решения

Я удаляю узлы из одного и того же дерева последовательно. Сначала удаляю лист 2. После этого у узла 4 остаётся только правый потомок 6, поэтому его можно использовать для второго случая. Третьим удаляю корень 8, у которого две непустые ветви.

Функция `delete` возвращает корень обновлённой ветви. У листа обе ссылки равны `None`, поэтому `return root.right` возвращает `None`. У узла с одним потомком возвращается этот потомок. При двух потомках я нахожу минимальный ключ справа, копирую его и удаляю его прежний узел.

Код `bst.py delete`

```
# Объявляю удаление ключа; возвращаю новый корень поддерева.
def delete(root, key):
    # Проверяю, пусто ли поддерево; в этом случае спуск продолжать нельзя.
    if root is None:
        # Возвращаю отсутствие узла; пустое поддерево остаётся пустым.
        return None
    # Если ключ меньше текущего, выбираю левое поддерево.
    if key < root.key:
        # Удаляю ключ слева и сохраняю обновлённую ссылку на эту ветвь.
        root.left = delete(root.left, key)
    # Если ключ больше текущего, выбираю правое поддерево.
    elif key > root.key:
        # Удаляю ключ справа и сохраняю обновлённую ссылку на эту ветвь.
        root.right = delete(root.right, key)
    # Если предыдущие условия не выполнены, выполняю эту оставшуюся ветвь.
    else:
        # Если слева пусто, узел можно заменить его правой ветвью.
        if root.left is None:
            # Возвращаю правого потомка вместо удаляемого узла; для листа это None.
            return root.right
        # Если справа пусто, вместо узла остаётся его левая ветвь.
        if root.right is None:
            # Возвращаю левого потомка вместо удаляемого узла.
            return root.left
        # При двух потомках начинаю поиск следующего ключа в правой ветви.
        successor = root.right
        # Пока есть левый потомок, можно найти ещё меньший ключ справа.
        while successor.left is not None:
```

```

        # Продвигаюсь влево к минимальному ключу правого поддерева.
        successor = successor.left
    # Копирую найденный ключ на место удаляемого ключа.
    root.key = successor.key
    # Удаляю старое вхождение перенесённого ключа из правой ветви.
    root.right = delete(root.right, successor.key)
# Возвращаю корень поддерева; вызывающий код сохраняет эту ссылку.
return root

```

Код main.py task5

```

# Объявляю демонстрацию задания 5; она выполняется при вызове.
def task5():
    # Строю новое BST из семи ключей; предыдущие задания его не изменяют.
    root = build_bst(KEYS)
    # Вывожу «Исходное дерево:» и соответствующие значения.
    print("Исходное дерево:")
    # Вызываю вывод структуры дерева; значения и связи при этом не меняются.
    show_tree(root)
    # По очереди получаю удаляемый ключ и подпись соответствующего случая.
    for key, case in [(2, "лист"), (4, "один потомок"), (8, "два потомка")]:
        # Вывожу «Удаляю →» и соответствующие значения.
        print("Удаляю", key, "→", case)
        # Вывожу «До:» и соответствующие значения.
        print("До:", inorder(root))
        # Сохраняю корень после удаления; особенно важно при удалении корня.
        root = delete(root, key)
        # Вызываю вывод структуры дерева; значения и связи при этом не меняются.
        show_tree(root)
        # Вывожу «После:» и соответствующие значения.
        print("После:", inorder(root))

```

Результат выполнения

Исходное дерево:

Корень: 8

Л: 4

Л: 2

П: 6

П: 12

Л: 10

П: 14

Удаляю 2 — лист

До: [2, 4, 6, 8, 10, 12, 14]

Корень: 8

Л: 4

П: 6

П: 12

Л: 10

П: 14

После: [4, 6, 8, 10, 12, 14]

Удаляю 4 — один потомок

До: [4, 6, 8, 10, 12, 14]

Корень: 8

Л: 6

П: 12

Л: 10

П: 14

После: [6, 8, 10, 12, 14]

Удаляю 8 — два потомка

До: [6, 8, 10, 12, 14]

Корень: 10

Л: 6

П: 12

П: 14

После: [6, 10, 12, 14]

Объяснение результата и вывод

После удаления 2 в дереве остаётся шесть ключей. После удаления 4 ссылка `root.left` указывает сразу на 6. При удалении 8 минимум правого поддерева равен 10: новый корневой ключ становится 10, а прежний узел 10 удаляется из левой ветви 12.

Итоговый inorder равен [6, 10, 12, 14], то есть порядок BST сохранился. Я вывожу схему до первого удаления и после каждого действия; схема предыдущего шага одновременно показывает состояние перед следующим удалением.

Вывод: удалять узел нужно с учётом его потомков. Для двух потомков недостаточно убрать ссылку на узел, иначе потеряются целые ветви. Удаление занимает $O(h)$, включая поиск следующего по величине ключа.

Задание 6 Влияние порядка вставки на высоту дерева

Условие задания

1. Постройте первое BST из последовательности 8, 4, 12, 2, 6, 10, 14.
2. Постройте второе BST из последовательности 2, 4, 6, 8, 10, 12, 14.
3. Реализуйте функцию `height(root)` и определите высоту обоих деревьев.
4. Выполните поиск последнего добавленного элемента в каждом дереве и сравните количество посещённых узлов.
5. Объясните, почему одинаковый набор ключей привёл к различной стоимости поиска.

Ход решения

Я строю два дерева с одинаковыми семью ключами. В первом порядок вставки распределяет элементы по обеим сторонам корня. Во втором каждое следующее число больше предыдущего, поэтому новая вставка всегда идёт вправо.

`height` возвращает -1 для пустой ветви. Для непустого узла она берёт большую из высот двух ветвей и добавляет 1. У листа получается $1 + \max(-1, -1) = 0$. В обоих деревьях последний добавленный ключ — 14.

Код `bst.py` `height`

```
# Объявляю вычисление высоты по рёбрам, а не по числу узлов.
def height(root):
    # Проверяю, пусто ли поддерево; в этом случае спуск продолжать нельзя.
    if root is None:
        # Для пустого дерева высота равна -1; тогда высота листа получается 0.
        return -1
    # Беру высоту более глубокой ветви и добавляю ребро от текущего узла.
    return 1 + max(height(root.left), height(root.right))
```

Код `main.py` `task6`

```
# Объявляю демонстрацию задания 6; она выполняется при вызове.
def task6():
    # Сравниваю исходный порядок вставки с уже отсортированным порядком.
    for values in [KEYS, [2, 4, 6, 8, 10, 12, 14]]:
        # Строю дерево именно из текущей последовательности values.
        root = build_bst(values)
        # Ищу последний добавленный ключ; индекс -1 обозначает последний элемент.
        node, visited = search(root, values[-1])
        # Вывожу «Вставка:» и соответствующие значения.
        print("Вставка:", values)
        # Вывожу «Высота: Поиск: Посещено:» и соответствующие значения.
        print("Высота:", height(root), "Поиск:", node.key, "Посещено:", visited)
```

```
# Вызываю вывод структуры дерева; значения и связи при этом не меняются.  
show_tree(root)
```

Результат выполнения

```
Вставка: [8, 4, 12, 2, 6, 10, 14]  
Высота: 2 Поиск: 14 Посещено: 3  
Корень: 8  
  Л: 4  
    Л: 2  
      П: 6  
        П: 12  
          Л: 10  
            П: 14  
Вставка: [2, 4, 6, 8, 10, 12, 14]  
Высота: 6 Поиск: 14 Посещено: 7  
Корень: 2  
  П: 4  
    П: 6  
      П: 8  
        П: 10  
          П: 12  
            П: 14
```

Объяснение результата и вывод

У первого дерева высота 2; поиск 14 проходит через 8, 12 и 14 и посещает 3 узла. У второго высота 6; поиск проходит через все ключи 2, 4, 6, 8, 10, 12, 14 и посещает 7 узлов.

Набор ключей одинаковый, но различается форма дерева. У сбалансированного BST высота порядка $\log n$, у цепочки — $n - 1$. Поэтому одинаковый алгоритм поиска может работать за $O(\log n)$ или за $O(n)$. Сама функция `height` просматривает все узлы и имеет время $O(n)$; это отдельный расчёт, а не поиск.

Порядок вставки	Высота в рёбрах	Посещений при поиске 14
8, 4, 12, 2, 6, 10, 14	2	3
2, 4, 6, 8, 10, 12, 14	6	7

Задание 7 Повороты AVL дерева

Условие задания

1. Реализуйте функции определения высоты узла и его баланс-фактора.
2. Реализуйте правый и левый повороты `rotate_right()` и `rotate_left()`.
3. Продемонстрируйте восстановление баланса для последовательностей 30, 20, 10 и 10, 20, 30.
4. Продемонстрируйте двойные повороты для последовательностей 30, 10, 20 и 10, 30, 20.

Ход решения

Я создаю четыре маленьких BST из последовательностей методички. Баланс-фактор равен $\text{height}(\text{left}) - \text{height}(\text{right})$. Значение 2 означает слишком высокую левую ветвь, -2 — слишком высокую правую. Для этих примеров достаточно локальных поворотов.

В правом повороте левый потомок становится новым корнем, а прежний корень — его правым потомком. Промежуточную ветвь `middle` сначала сохраняю, затем присоединяю к прежнему корню. Левый поворот выполняет симметричные действия. Сохранение `middle` нужно и для более крупных деревьев, где эта ветвь не пустая.

Для LR сначала поворачиваю левую ветвь влево, затем корень вправо. Для RL сначала поворачиваю правую ветвь вправо, затем корень влево. Полная автоматическая вставка и удаление AVL в задании не требуются: здесь я показываю сами повороты.

Схема

LL: 30 → слева 20 → слева 10	правый поворот
RR: 10 → справа 20 → справа 30	левый поворот
LR: 30 → слева 10 → справа 20	левый, затем правый
RL: 10 → справа 30 → слева 20	правый, затем левый

После любого из четырёх случаев:

```
      20
     /  \
    10   30
```

Код bst.py balance_factor

```
# Объявляю вычисление разности высот левого и правого поддеревьев.
def balance_factor(root):
    # Проверяю, пусто ли поддерево; в этом случае спуск продолжать нельзя.
    if root is None:
        # Для отсутствующего узла принимаю баланс-фактор равным нулю.
        return 0
    # Вычитаю высоту правой ветви из высоты левой; это баланс-фактор.
    return height(root.left) - height(root.right)
```

Код bst.py rotate_right

```
# Объявляю правый поворот; у root должен существовать левый потомок.
def rotate_right(root):
    # Левый потомок станет новым корнем при правом повороте.
    new_root = root.left
    # Временно сохраняю среднюю ветвь, чтобы не потерять её при повороте.
    middle = new_root.right
    # Прежний корень становится правым потомком нового корня.
    new_root.right = root
    # Средняя ветвь становится левой ветвью прежнего корня.
    root.left = middle
    # Возвращаю новый корень; после поворота его нужно сохранить.
    return new_root
```

Код bst.py rotate_left

```
# Объявляю левый поворот; у root должен существовать правый потомок.
def rotate_left(root):
    # Правый потомок станет новым корнем при левом повороте.
    new_root = root.right
    # Сохраняю среднюю левую ветвь будущего корня перед изменением связей.
    middle = new_root.left
    # Прежний корень становится левым потомком нового корня.
    new_root.left = root
    # Средняя ветвь становится правой ветвью прежнего корня.
    root.right = middle
    # Возвращаю новый корень; после поворота его нужно сохранить.
    return new_root
```

Код main.py task7

```
# Объявляю демонстрацию задания 7; она выполняется при вызове.
def task7():
    # Начинаю список случаев: левый перекос LL и правый перекос RR.
    cases = [("LL", [30, 20, 10]), ("RR", [10, 20, 30]),
             # Дополняю список двумя зигзагообразными перекосами LR и RL.
             ("LR", [30, 10, 20]), ("RL", [10, 30, 20])]
    # Для каждого случая беру его название и порядок вставки трёх ключей.
    for case, values in cases:
        # Строю дерево именно из текущей последовательности values.
        root = build_bst(values)
        # Вывожу «Вставка: BF до:» и соответствующие значения.
        print(case, "Вставка:", values, "BF до:", balance_factor(root))
        # Вызываю вывод структуры дерева; значения и связи при этом не меняются.
        show_tree(root)
        # Левая ветвь растёт влево; для LL нужен один правый поворот.
        if case == "LL":
            # Делаю правый поворот и сохраняю возвращённый новый корень.
            root = rotate_right(root)
        # Правая ветвь растёт вправо; для RR нужен один левый поворот.
        elif case == "RR":
            # Делаю левый поворот и сохраняю новый корень.
            root = rotate_left(root)
        # Левая ветвь растёт вправо; для LR сначала исправляю её зигзаг.
        elif case == "LR":
```

```

        # Для LR делаю левый поворот вокруг левого потомка корня.
        root.left = rotate_left(root.left)
        # Делаю правый поворот и сохраняю возвращённый новый корень.
        root = rotate_right(root)
    # Если предыдущие условия не выполнены, выполняю эту оставшуюся ветвь.
    else:
        # Для RL делаю правый поворот вокруг правого потомка корня.
        root.right = rotate_right(root.right)
        # Делаю левый поворот и сохраняю новый корень.
        root = rotate_left(root)
    # Вывожу «После поворотов, BF:» и соответствующие значения.
    print("После поворотов, BF:", balance_factor(root))
    # Вызываю вывод структуры дерева; значения и связи при этом не меняются.
    show_tree(root)
    # Вывожу «Inorder:» и соответствующие значения.
    print("Inorder:", inorder(root))

```

Результат выполнения

```

LL Вставка: [30, 20, 10] BF до: 2
Корень: 30
    Л: 20
        Л: 10
После поворотов, BF: 0
Корень: 20
    Л: 10
    П: 30
Inorder: [10, 20, 30]
RR Вставка: [10, 20, 30] BF до: -2
Корень: 10
    П: 20
        П: 30
После поворотов, BF: 0
Корень: 20
    Л: 10
    П: 30
Inorder: [10, 20, 30]
LR Вставка: [30, 10, 20] BF до: 2
Корень: 30
    Л: 10
        П: 20
После поворотов, BF: 0
Корень: 20
    Л: 10
    П: 30
Inorder: [10, 20, 30]
RL Вставка: [10, 30, 20] BF до: -2
Корень: 10
    П: 30
        Л: 20
После поворотов, BF: 0
Корень: 20
    Л: 10
    П: 30
Inorder: [10, 20, 30]

```

Объяснение результата и вывод

Во всех четырёх случаях после поворотов корнем становится 20, левым потомком 10, правым — 30. Баланс-фактор корня меняется с 2 или -2 на 0. Inorder во всех случаях остаётся [10, 20, 30], поэтому перестройка не нарушает правило BST.

Один поворот меняет фиксированное число ссылок и выполняется за $O(1)$. Но моя учебная `balance_factor` пересчитывает высоты рекурсивно и может затратить $O(n)$. Это не реализация полного AVL с сохранёнными высотами; в промышленном AVL служебные высоты поддерживаются при изменениях.

Вывод: баланс восстанавливается перестройкой связей без повторной вставки всех значений. Поворот возвращает новый корень, поэтому присваивание `root = rotate_right(root)` обязательно.

Случай	BF до	Повороты	Корень после	BF после
LL	2	Правый	20	0
RR	-2	Левый	20	0
LR	2	Левый, правый	20	0
RL	-2	Правый, левый	20	0

Задание 8 Наивный запрос суммы диапазона

Условие задания

1. Создайте список не менее чем из 100 000 целых чисел. Допускается использование модуля `random`.
2. Реализуйте функцию `range_sum_naive(data, left, right)`, вычисляющую сумму элементов с индексами от `left` до `right` включительно обычным циклом.
3. Сформируйте не менее 1000 случайных диапазонов и выполните запросы для каждого из них.
4. Измерьте суммарное время выполнения запросов с помощью `time.perf_counter()`.

Ход решения

Я создаю 100000 целых чисел от -100 до 100 и 1000 случайных диапазонов. Использую отдельный генератор `random.Random(42)`. При повторном вызове `make_experiment` с тем же `seed` получаются те же числа и границы. Если две случайные границы выбраны в обратном порядке, меняю их местами.

`range_sum_naive` проходит все индексы от `left` до `right` включительно и накапливает сумму. Измерение начинается после подготовки данных. Внутри замера находятся только цикл запросов, вызов функции и сохранение ответа в список. Печать, контрольные `sum()` и генерация случайных чисел находятся вне замера.

Код `ranges.py range_sum_naive`

```
# Объявляю обычный подсчёт суммы на включительном диапазоне.
def range_sum_naive(data, left, right):
    # Начальная сумма равна нулю; затем к ней будут прибавляться значения.
    result = 0
    # Перебираю индексы от left до right; +1 включает правую границу.
    for i in range(left, right + 1):
        # Прибавляю значение очередного элемента диапазона к сумме.
        result += data[i]
    # Возвращаю накопленный результат вызывающему коду.
    return result
```

Код `main.py make_experiment`

```
# Объявляю создание воспроизводимых данных и диапазонов для замеров.
def make_experiment():
    # Создаю отдельный генератор с seed 42; повторный вызов даёт те же данные.
    generator = random.Random(42)
```

```

# Создаю пустой список исходных значений.
data = []
# Выполняю 100000 шагов; подчёркивание в числе служит только для чтения.
for i in range(100_000):
    # Добавляю случайное целое число от -100 до 100 включительно.
    data.append(generator.randint(-100, 100))
# Создаю список запросов; каждый запрос будет парой границ.
queries = []
# Выполняю 1000 шагов для создания запросов или измерений.
for i in range(1000):
    # Выбираю допустимый левый индекс от 0 до n - 1.
    left = generator.randrange(len(data))
    # Выбираю второй допустимый индекс от 0 до n - 1.
    right = generator.randrange(len(data))
    # Проверяю, не оказались ли случайные границы в обратном порядке.
    if left > right:
        # Меняю две границы местами, чтобы left не превышал right.
        left, right = right, left
    # Сохраняю включительный диапазон как пару индексов.
    queries.append((left, right))
# Возвращаю массив и список запросов для дальнейшего измерения.
return data, queries

```

Код main.py task8

```

# Объявляю демонстрацию задания 8; она выполняется при вызове.
def task8():
    # Получаю воспроизводимые данные и запросы до начала замера.
    data, queries = make_experiment()
    # Создаю список для сохранения ответов на запросы.
    results = []
    # Запоминаю время начала только измеряемого этапа.
    started = time.perf_counter()
    # Перебираю заранее подготовленные пары границ запросов.
    for left, right in queries:
        # Считаю сумму обычным циклом и сохраняю ответ.
        results.append(range_sum_naive(data, left, right))
    # Вычитаю начало из текущего времени; получаю длительность в секундах.
    elapsed = time.perf_counter() - started
    # Вывожу «Элементов: Запросов: Seed: 42» и соответствующие значения.
    print("Элементов:", len(data), "Запросов:", len(queries), "Seed: 42")
    # Вывожу «Время обычного цикла, с:» и соответствующие значения.
    print("Время обычного цикла, с:", round(elapsed, 6))
    # Вывожу три первых запроса, чтобы результат можно было проверить вручную.
    for i in range(3):
        # Из пары запроса получаю левую и правую границы.
        left, right = queries[i]
        # Вне замера проверяю сумму среза; правая граница среза не включается.
        expected = sum(data[left:right + 1])
        # Вывожу «Диапазон: Сумма: sum():» и соответствующие значения.
        print("Диапазон:", queries[i], "Сумма:", results[i], "sum():", expected)
    # Вывожу «Контрольная сумма 1000 ответов:» и соответствующие значения.
    print("Контрольная сумма 1000 ответов:", sum(results))

```

Результат выполнения

```

Элементов: 100000 Запросов: 1000 Seed: 42
Время обычного цикла, с: 2.897749
Диапазон: (83554, 98087) Сумма: -3510 sum(): -3510
Диапазон: (11986, 56837) Сумма: -4888 sum(): -4888
Диапазон: (25527, 50084) Сумма: -2191 sum(): -2191
Контрольная сумма 1000 ответов: 3954384

```

Объяснение результата и вывод

В результате указано суммарное время всех 1000 запросов. Для первых трёх диапазонов ответы совпали с `sum()` по соответствующему срезу. Контрольная сумма всех ответов помогает проверить, что при повторном запуске набор запросов не изменился.

Один запрос имеет время $O(\text{right} - \text{left} + 1)$, в худшем случае $O(n)$. Дополнительная память самой функции суммы $O(1)$. При q запросах худшее суммарное время $O(nq)$. В этом опыте я дополнительно храню список из q ответов, который занимает $O(q)$.

Задание 9 Префиксные суммы

Условие задания

1. Реализуйте функцию `build_prefix(data)`, создающую массив префиксных сумм.
2. Реализуйте функцию `range_sum_prefix(prefix, left, right)`.
3. Выполните для тех же диапазонов, что использовались в предыдущем задании, запросы через префиксные суммы.
5. Сравните время выполнения запросов и отдельно зафиксируйте время предварительного построения `prefix`.
6. Объясните, почему после изменения одного элемента исходного массива часть префиксных сумм становится некорректной.

Ход решения

Я вновь вызываю `make_experiment: seed 42` воспроизводит тот же массив и те же 1000 диапазонов задания 8. Наивный метод измеряю повторно в этом задании, чтобы иметь отдельное сравнение с префиксным методом в одном запуске. Результаты обоих методов сохраняю в списки и сравниваю целиком.

В `prefix[0]` находится 0. `prefix[i + 1]` хранит сумму элементов `data[0]..data[i]`. Для включительного диапазона `[left, right]` беру `prefix[right + 1] - prefix[left]`. Время построения и время запросов измеряются отдельно, также вывожу их сумму.

Для примера `A = [5, 2, 7, 3, 6, 1, 4, 8]` префиксы равны `[0, 5, 7, 14, 17, 23, 24, 28, 36]`. Запрос `[2, 6]` равен `prefix[7] - prefix[2] = 28 - 7 = 21`.

Схема

A:	5	2	7	3	6	1	4	8	
Индекс:	0	1	2	3	4	5	6	7	
prefix:	0	5	7	14	17	23	24	28	36
[2, 6]:	<code>prefix[7] - prefix[2] = 28 - 7 = 21</code>								

Код `ranges.py` `build_prefix`

```
# Объявляю построение списка сумм от начала исходного массива.
def build_prefix(data):
    # Создаю n + 1 нулей; дополнительная позиция хранит пустой префикс.
    prefix = [0] * (len(data) + 1)
    # Перебираю внешние индексы исходного массива от 0 до n - 1.
    for i in range(len(data)):
        # К сумме первых i элементов прибавляю элемент data[i].
        prefix[i + 1] = prefix[i] + data[i]
```

```
# Возвращаю готовый список префиксных сумм.
return prefix
```

Код ranges.py range_sum_prefix

```
# Объявляю быстрый запрос суммы через готовые префиксы.
def range_sum_prefix(prefix, left, right):
    # Из суммы до right включительно вычитаю сумму элементов до left.
    return prefix[right + 1] - prefix[left]
```

Код main.py task9

```
# Объявляю демонстрацию задания 9; она выполняется при вызове.
def task9():
    # Получаю воспроизводимые данные и запросы до начала замера.
    data, queries = make_experiment()
    # Готовлю список эталонных ответов, полученных обычным циклом.
    naive_results = []
    # Запоминаю время начала только измеряемого этапа.
    started = time.perf_counter()
    # Перебираю заранее подготовленные пары границ запросов.
    for left, right in queries:
        # Повторно измеряю наивный метод на тех же данных и сохраняю ответ.
        naive_results.append(range_sum_naive(data, left, right))
    # Запоминаю суммарное время 1000 наивных запросов.
    naive_time = time.perf_counter() - started
    # Запоминаю время начала только измеряемого этапа.
    started = time.perf_counter()
    # Строю префиксные суммы; время этого этапа измеряется отдельно.
    prefix = build_prefix(data)
    # Запоминаю длительность предварительного построения префиксов.
    build_time = time.perf_counter() - started
    # Готовлю отдельный список ответов префиксного метода.
    prefix_results = []
    # Запоминаю время начала только измеряемого этапа.
    started = time.perf_counter()
    # Перебираю заранее подготовленные пары границ запросов.
    for left, right in queries:
        # Вычисляю очередную сумму двумя обращениями к готовым префиксам.
        prefix_results.append(range_sum_prefix(prefix, left, right))
    # Запоминаю время только 1000 запросов к уже построенным префиксам.
    query_time = time.perf_counter() - started
    # Вывожу «Данные и 1000 диапазонов совпадают с заданием 8: seed 42» и соответствующие
    # значения.
    print("Данные и 1000 диапазонов совпадают с заданием 8: seed 42")
    # Вывожу «Обычный цикл, с:» и соответствующие значения.
    print("Обычный цикл, с:", round(naive_time, 6))
    # Вывожу «Построение prefix, с:» и соответствующие значения.
    print("Построение prefix, с:", round(build_time, 6))
    # Вывожу «1000 запросов prefix, с:» и соответствующие значения.
    print("1000 запросов prefix, с:", round(query_time, 6))
    # Вывожу «Построение и запросы вместе, с:» и соответствующие значения.
    print("Построение и запросы вместе, с:", round(build_time + query_time, 6))
    # Вывожу «Все 1000 ответов совпали:» и соответствующие значения.
    print("Все 1000 ответов совпали:", naive_results == prefix_results)
    # Создаю копию маленького массива, чтобы не менять общий EXAMPLE.
    small = EXAMPLE.copy()
    # Сохраняю префиксы для массива до изменения элемента.
    old_prefix = build_prefix(small)
    # Заменяю третий элемент; старые префиксы сами от этого не меняются.
    small[2] = 10
    # Вывожу «После замены A[2] = 10, старый prefix:» и соответствующие значения.
    print("После замены A[2] = 10, старый prefix:", range_sum_prefix(old_prefix, 2, 6))
    # Вывожу «Правильная сумма:» и соответствующие значения.
    print("Правильная сумма:", sum(small[2:7]))
    # Вывожу «Новые префиксы:» и соответствующие значения.
    print("Новые префиксы:", build_prefix(small))
```

Результат выполнения

Данные и 1000 диапазонов совпадают с заданием 8: seed 42
Обычный цикл, с: 2.791033
Построение prefix, с: 0.019037
1000 запросов prefix, с: 0.000404
Построение и запросы вместе, с: 0.01944
Все 1000 ответов совпали: True
После замены $A[2] = 10$, старый prefix: 21
Правильная сумма: 24
Новые префиксы: [0, 5, 7, 17, 20, 26, 27, 31, 39]

Сравнение времени на одинаковых запросах

Этап	Время в секундах	Что измеряется
Обычный цикл	2.791033	1000 запросов
Построение prefix	0.019037	Подготовка префиксов
Запросы prefix	0.000404	1000 готовых запросов
Prefix вместе	0.01944	Построение и 1000 запросов

Объяснение результата и вывод

Все 1000 ответов префиксного метода совпали с обычным циклом. Построение имеет время $O(n)$, каждый готовый запрос — $O(1)$. Для q запросов суммарно получается $O(n + q)$, а список префиксов занимает $O(n)$ памяти.

После замены $A[2]$ с 7 на 10 старые префиксы всё ещё дают 21 для [2, 6], хотя правильная сумма стала 24. Разница равна 3. Необходимо изменить $prefix[3]$ и все следующие значения; в данном примере новые префиксы заканчиваются числом 39.

Время является реальным замером конкретного запуска и меняется при другой нагрузке. Времена включают вызов функции и добавление ответа в список у обоих методов. Вывод о сложности следует из числа действий алгоритма; по одному замеру нельзя считать коэффициент ускорения постоянным.

Задание 10 Дерево Фенвика

Условие задания

1. Реализуйте класс FenwickTree.
2. Добавьте метод `update(index, delta)` для точечного изменения значения.
3. Добавьте метод `prefix_sum(index)` для вычисления префиксной суммы.
4. Добавьте метод `range_sum(left, right)`, использующий разность двух префиксных сумм.
5. Постройте структуру для тестового массива и сравните несколько запросов с результатом функции `sum()`.
7. В отчёте объясните назначение операции $i \& -i$.

Ход решения

Я строю FenwickTree для массива из лекции. Во внешних методах индекс начинается с нуля, а внутри `update` и `prefix_sum` выполняется $i = \text{index} + 1$. Внутренний `tree[0]` не хранит полезный блок. Начальные значения добавляю последовательными вызовами `update`.

Выражение $i \& -i$ выделяет младший установленный бит положительного i . Полученное число называется `lowbit` и задаёт длину блока `tree[i]`. Например, $\text{lowbit}(6) = 2$, поэтому внутренний `tree[6]` хранит сумму позиций 5 и 6, то есть внешних `data[4]` и `data[5]`. Эти значения равны 6 и 1, сумма блока равна 7.

При подсчёте префикса вычитаю `lowbit`: блоки не пересекаются и вместе покрывают нужное начало массива. При обновлении прибавляю `lowbit`: перехожу к более крупным блокам, содержащим изменённый элемент. `range_sum` вычитает префикс до `left - 1` из префикса до `right`.

Для замены `A[2]` с 7 на 10 вычисляю $\text{delta} = 10 - 7 = 3$. Сам исходный список меняю отдельно. Метод `update(2, 3)` прибавляет 3, а не записывает число 3 вместо старого значения.

Внутренние блоки Фенвика

i	i & -i	Внутренний блок	Внешние индексы	Сумма
1	1	[1, 1]	[0, 0]	5
2	2	[1, 2]	[0, 1]	7
3	1	[3, 3]	[2, 2]	7
4	4	[1, 4]	[0, 3]	17
5	1	[5, 5]	[4, 4]	6
6	2	[5, 6]	[4, 5]	7
7	1	[7, 7]	[6, 6]	4
8	8	[1, 8]	[0, 7]	36

В двоичном виде $6 = 0110$. Для наглядного четырёхбитного примера дополнительный код -6 равен 1010 . Побитовое И оставляет 0010 , то есть 2. Python не ограничен четырьмя битами, но для положительного i операция $i \& -i$ даёт тот же младший установленный бит.

Код ranges.py FenwickTree

```
# Объявляю класс дерева Фенвика для сумм и прибавления delta.
class FenwickTree:
    # Создаю объект и заполняю его начальные поля; self означает этот объект.
    def __init__(self, data):
        # Запоминаю количество исходных элементов в поле n объекта.
        self.n = len(data)
        # Создаю массив Фенвика; индекс 0 не используется для блоков.
        self.tree = [0] * (self.n + 1)
        # Перебираю все настоящие элементы исходного массива.
        for i in range(self.n):
            # Добавляю исходное значение в блоки Фенвика через обычное обновление.
            self.update(i, data[i])

    # Объявляю точечное обновление; смысл второго аргумента зависит от класса.
    def update(self, index, delta):
        # Проверяю индекс: разрешены только значения от 0 до n - 1.
        if index < 0 or index >= self.n:
            # Отклоняю неверный индекс, чтобы не обращаться вне структуры.
            raise IndexError("Индекс вне массива")
        # Перевожу внешний индекс с нуля во внутренний индекс Фенвика с единицы.
        i = index + 1
        # Обновляю блоки, пока внутренний индекс остаётся внутри массива.
        while i <= self.n:
            # Прибавляю изменение delta к сумме текущего блока.
            self.tree[i] += delta
            # Прибавляю lowbit и перехожу к следующему блоку, содержащему элемент.
            i += i & -i

    # Объявляю сумму от внешнего индекса 0 до index включительно.
    def prefix_sum(self, index):
        # Разрешаю обычные индексы и -1 для суммы пустого префикса.
        if index < -1 or index >= self.n:
            # Отклоняю неверный индекс, чтобы не обращаться вне структуры.
            raise IndexError("Индекс вне массива")
        # Перевожу внешний индекс с нуля во внутренний индекс Фенвика с единицы.
        i = index + 1
```

```

# Начальная сумма равна нулю; затем к ней будут прибавляться значения.
result = 0
# Собираю префикс из блоков до достижения внутреннего индекса 0.
while i > 0:
    # Прибавляю сохранённую сумму текущего блока Фенвика.
    result += self.tree[i]
    # Вычитаю lowbit и перехожу к предыдущему непересекающемуся блоку.
    i -= i & -i
# Возвращаю накопленный результат вызывающему коду.
return result

# Объявляю сумму диапазона как разность двух префиксов.
def range_sum(self, left, right):
    # Проверяю границы и их порядок; диапазон должен быть непустым.
    if left < 0 or right >= self.n or left > right:
        # Сообщаю об ошибке границ вместо расчёта неверного результата.
        raise IndexError("Некорректный диапазон")
    # Из суммы до right вычитаю сумму до left - 1; при left = 0 она равна 0.
    return self.prefix_sum(right) - self.prefix_sum(left - 1)

```

Код main.py task10

```

# Объявляю демонстрацию задания 10; она выполняется при вызове.
def task10():
    # Создаю независимую копию примера, чтобы обновления не влияли на другие задания.
    data = EXAMPLE.copy()
    # Строю дерево Фенвика по текущим исходным значениям.
    tree = FenwickTree(data)
    # Вывожу «Массив:» и соответствующие значения.
    print("Массив:", data)
    # Вывожу «tree[1..n]:» и соответствующие значения.
    print("tree[1..n]:", tree.tree[1:])
    # Вывожу «Префикс индексов 0..6:» и соответствующие значения.
    print("Префикс индексов 0..6:", tree.prefix_sum(6))
    # Проверяю шесть заранее заданных диапазонов разной длины.
    for left, right in CHECK_RANGES:
        # Вывожу «Фенвик: sum():» и соответствующие значения.
        print((left, right), "Фенвик:", tree.range_sum(left, right),
              # Продолжение print: рядом вывожу обычную сумму среза для проверки дерева.
              "sum():", sum(data[left:right + 1]))
    # Вычисляю изменение: новое значение 10 минус старое значение 7.
    delta = 10 - data[2]
    # Заменяю элемент в исходном списке; структуру нужно обновить отдельно.
    data[2] = 10
    # Сообщаю Фенвику прибавку delta для внешнего индекса 2.
    tree.update(2, delta)
    # Вывожу «Замена A[2]: 7 -> 10; delta =» и соответствующие значения.
    print("Замена A[2]: 7 -> 10; delta =", delta)
    # Вывожу «tree[1..n]:» и соответствующие значения.
    print("tree[1..n]:", tree.tree[1:])
    # Вывожу «После обновления [2, 6]: sum():» и соответствующие значения.
    print("После обновления [2, 6]:", tree.range_sum(2, 6), "sum():", sum(data[2:7]))

```

Результат выполнения

```

Массив: [5, 2, 7, 3, 6, 1, 4, 8]
tree[1..n]: [5, 7, 7, 17, 6, 7, 4, 36]
Префикс индексов 0..6: 28
(0, 0) Фенвик: 5 sum(): 5
(0, 7) Фенвик: 36 sum(): 36
(2, 6) Фенвик: 21 sum(): 21
(1, 5) Фенвик: 19 sum(): 19
(7, 7) Фенвик: 8 sum(): 8
(3, 4) Фенвик: 9 sum(): 9
Замена A[2]: 7 -> 10; delta = 3
tree[1..n]: [5, 7, 10, 20, 6, 7, 4, 39]
После обновления [2, 6]: 24 sum(): 24

```

Объяснение результата и вывод

Начальные внутренние блоки равны [5, 7, 7, 17, 6, 7, 4, 36]. Префикс внешних индексов 0..6 равен $4 + 7 + 17 = 28$: внутренние позиции идут $7 \rightarrow 6 \rightarrow 4 \rightarrow 0$. Это сумма первых семи элементов, а не 27.

При обновлении внешнего индекса 2 внутренние позиции идут $3 \rightarrow 4 \rightarrow 8$. Их суммы увеличиваются на 3. Запрос [2, 6] после изменения равен 24 и совпадает с обычным `sum()`. Для `left = 0` используется `prefix_sum(-1)`, которая возвращает сумму пустого префикса 0.

В этой реализации построение через n вызовов `update` имеет время $O(n \log n)$, как показано в лекции. Префиксный запрос, запрос диапазона и точечное обновление имеют время $O(\log n)$. Память составляет $O(n)$.

Задание 11 Дерево отрезков для суммы

Условие задания

1. Реализуйте класс `SegmentTree` для хранения суммы элементов.
2. Реализуйте построение дерева по исходному массиву.
3. Реализуйте метод `query(left, right)`, возвращающий сумму элементов заданного диапазона.
4. Реализуйте метод `update(index, value)`, изменяющий одно значение исходного массива и пересчитывающий необходимые узлы.
5. Проверьте не менее пяти диапазонных запросов сравнением с обычным `sum()`.

Ход решения

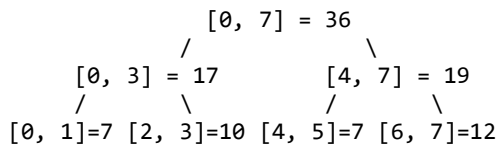
Я представляю дерево отрезков массивами. S — ближайшая степень двойки, не меньшая n . Лист элемента `data[i]` находится в позиции $S + i$. Корень имеет индекс 1, левый потомок узла i — $2 * i$, правый — $2 * i + 1$. Для восьми элементов $S = 8$.

Сначала записываю элементы в листья. Затем от $S - 1$ до 1 пересчитываю родителей по детям. Так каждый внутренний узел получает сумму своего диапазона. Полный класс ниже уже включает расширение задания 12: вместе с суммой хранятся минимум и максимум. В этом задании использую только метод `query`, возвращающий первую характеристику — сумму.

Запрос переводит границы в листья и поднимается по уровням. Если левая граница нечётная, она является правым потомком: левый сосед уже вне запроса, поэтому этот узел беру отдельно. Если правая граница чётная, она является левым потомком и тоже берётся отдельно. Учтённые края убираю, а оставшуюся часть перевозжу к родителям.

На каждом уровне учитывается не более двух крайних узлов. Большие полностью подходящие сегменты используются целиком; элементы вне запроса не добавляются. Это та же идея разбиения диапазона, которую показывает лекция.

Схема



Запрос [2, 6]: [2, 3] + [4, 5] + [6, 6]
Сумма: 10 + 7 + 4 = 21

Код ranges.py SegmentTree

```
# Объявляю класс дерева отрезков для суммы, минимума и максимума.
class SegmentTree:
    # Создаю объект и заполняю его начальные поля; self означает этот объект.
    def __init__(self, data):
        # Запоминаю количество исходных элементов в поле n объекта.
        self.n = len(data)
        # Проверяю, что исходный массив для дерева отрезков не пуст.
        if self.n == 0:
            # Отклоняю пустой массив: в этой реализации нужен хотя бы один лист.
            raise ValueError("Массив должен быть непустым")
        # Начинаю подбор числа листьев с первой степени двойки, то есть с 1.
        self.S = 1
        # Увеличиваю число листьев, пока для всех элементов не хватит места.
        while self.S < self.n:
            # Удваиваю S; так получаю следующую степень двойки.
            self.S *= 2
        # Создаю массив сумм; листья начинаются с S, корень находится в позиции 1.
        self.tree = [0] * (2 * self.S)
        # Для пустых листьев минимум равен плюс бесконечности и не портит ответ.
        self.minimum = [float("inf")] * (2 * self.S)
        # Для пустых листьев максимум равен минус бесконечности и не портит ответ.
        self.maximum = [float("-inf")] * (2 * self.S)
        # Перебираю все настоящие элементы исходного массива.
        for i in range(self.n):
            # Нахожу внутреннюю позицию листа для внешнего элемента data[i].
            position = self.S + i
            # В листе сумма равна единственному исходному элементу.
            self.tree[position] = data[i]
            # Минимум листа равен его единственному элементу.
            self.minimum[position] = data[i]
            # Максимум листа тоже равен его единственному элементу.
            self.maximum[position] = data[i]
        # Перебираю внутренние узлы снизу вверх, от S - 1 до корня 1.
        for i in range(self.S - 1, 0, -1):
            # Пересчитываю сумму, минимум и максимум узла по его потомкам.
            self.pull(i)

    # Объявляю пересчёт трёх характеристик узла i по его двум потомкам.
    def pull(self, i):
        # Сумма родителя равна сумме левого и правого потомков.
        self.tree[i] = self.tree[2 * i] + self.tree[2 * i + 1]
        # Минимум родителя выбираю из минимумов двух потомков.
        self.minimum[i] = min(self.minimum[2 * i], self.minimum[2 * i + 1])
        # Максимум родителя выбираю из максимумов двух потомков.
        self.maximum[i] = max(self.maximum[2 * i], self.maximum[2 * i + 1])

    # Объявляю запрос только суммы; он использует общий запрос характеристик.
    def query(self, left, right):
        # Беру первый элемент тройки характеристик; в нём находится сумма.
        return self.query_stats(left, right)[0]

    # Объявляю запрос суммы, минимума и максимума включительного диапазона.
    def query_stats(self, left, right):
        # Проверяю границы и их порядок; диапазон должен быть непустым.
        if left < 0 or right >= self.n or left > right:
```

```

        # Сообщаю об ошибке границ вместо расчёта неверного результата.
        raise IndexError("Некорректный диапазон")
    # Перевожу левую внешнюю границу в позицию листа дерева.
    left += self.S
    # Перевожу правую внешнюю границу в позицию листа дерева.
    right += self.S
    # Начальная сумма выбранных сегментов равна нулю.
    total = 0
    # Начальный минимум задаю бесконечностью; любое настоящее число меньше.
    smallest = float("inf")
    # Начальный максимум задаю минус бесконечностью; настоящее число больше.
    largest = float("-inf")
    # Обрабатываю оставшийся диапазон, пока его границы не пересеклись.
    while left <= right:
        # Нечётная левая граница – правый потомок; беру его отдельно от соседа.
        if left % 2 == 1:
            # Прибавляю сумму целиком подходящего сегмента на левой границе.
            total += self.tree[left]
            # Сравниваю текущий минимум с минимумом выбранного левого сегмента.
            smallest = min(smallest, self.minimum[left])
            # Сравниваю текущий максимум с максимумом выбранного левого сегмента.
            largest = max(largest, self.maximum[left])
            # Убираю уже учтённый левый сегмент из оставшегося диапазона.
            left += 1
        # Чётная правая граница – левый потомок; учитываю его отдельно от соседа.
        if right % 2 == 0:
            # Прибавляю сумму целиком подходящего сегмента на правой границе.
            total += self.tree[right]
            # Обновляю минимум с учётом выбранного правого сегмента.
            smallest = min(smallest, self.minimum[right])
            # Обновляю максимум с учётом выбранного правого сегмента.
            largest = max(largest, self.maximum[right])
            # Убираю уже учтённый правый сегмент из оставшегося диапазона.
            right -= 1
        # Перехожу от левой границы к её родителю; // означает целое деление.
        left //= 2
        # Перехожу от правой границы к её родителю на следующем уровне.
        right //= 2
    # Возвращаю тройку: сумма, минимум и максимум найденного диапазона.
    return total, smallest, largest

# Объявляю точечное обновление; смысл второго аргумента зависит от класса.
def update(self, index, value):
    # Проверяю индекс: разрешены только значения от 0 до n - 1.
    if index < 0 or index >= self.n:
        # Отклоняю неверный индекс, чтобы не обращаться вне структуры.
        raise IndexError("Индекс вне массива")
    # Нахожу лист, в котором хранится изменяемый исходный элемент.
    i = self.S + index
    # Записываю новое значение в сумму листа; это замена, а не прибавление.
    self.tree[i] = value
    # После замены минимум листа равен новому значению.
    self.minimum[i] = value
    # После замены максимум листа равен новому значению.
    self.maximum[i] = value
    # Поднимаюсь к родителю текущего узла через целочисленное деление на 2.
    i //= 2
    # Пересчитываю всех предков изменённого листа, включая корень 1.
    while i >= 1:
        # Пересчитываю сумму, минимум и максимум узла по его потомкам.
        self.pull(i)
        # Поднимаюсь к родителю текущего узла через целочисленное деление на 2.
        i //= 2

```

Код main.py task11

```

# Объявляю демонстрацию задания 11; она выполняется при вызове.
def task11():
    # Создаю независимую копию примера, чтобы обновления не влияли на другие задания.
    data = EXAMPLE.copy()

```

```

# Строю дерево отрезков для текущего массива.
tree = SegmentTree(data)
# Вывожу «Массив: S:» и соответствующие значения.
print("Массив:", data, "S:", tree.S)
# Вывожу «Суммы в вершинах tree[1..]:» и соответствующие значения.
print("Суммы в вершинах tree[1..]:", tree.tree[1:])
# Проверяю шесть заранее заданных диапазонов разной длины.
for left, right in CHECK_RANGES:
    # Вывожу «Дерево: sum():» и соответствующие значения.
    print((left, right), "Дерево:", tree.query(left, right),
          # Продолжение print: рядом вывожу обычную сумму среза для проверки дерева.
          "sum():", sum(data[left:right + 1]))
# Заменяю значение с внешним индексом 2 с 7 на 6.
data[2] = 6
# Передаю дереву отрезков новое значение 6, а не величину изменения.
tree.update(2, 6)
# Вывожу «Замена A[2]: 7 -> 6» и соответствующие значения.
print("Замена A[2]: 7 -> 6")
# Проверяю шесть заранее заданных диапазонов разной длины.
for left, right in CHECK_RANGES:
    # Вывожу «После: sum():» и соответствующие значения.
    print((left, right), "После:", tree.query(left, right),
          # Продолжение print: рядом вывожу обычную сумму среза для проверки дерева.
          "sum():", sum(data[left:right + 1]))

```

Результат выполнения

```
Массив: [5, 2, 7, 3, 6, 1, 4, 8] S: 8
Суммы в вершинах tree[1..]: [36, 17, 19, 7, 10, 7, 12, 5, 2, 7, 3, 6, 1, 4, 8]
(0, 0) Дерево: 5 sum(): 5
(0, 7) Дерево: 36 sum(): 36
(2, 6) Дерево: 21 sum(): 21
(1, 5) Дерево: 19 sum(): 19
(7, 7) Дерево: 8 sum(): 8
(3, 4) Дерево: 9 sum(): 9
Замена A[2]: 7 -> 6
(0, 0) После: 5 sum(): 5
(0, 7) После: 35 sum(): 35
(2, 6) После: 20 sum(): 20
(1, 5) После: 18 sum(): 18
(7, 7) После: 8 sum(): 8
(3, 4) После: 9 sum(): 9
```

Объяснение результата и вывод

Шесть запросов совпали с `sum()`: включены оба крайних одиночных элемента, весь массив и промежуточные диапазоны. Для `[2, 6]` используются сегменты `[2, 3]`, `[4, 5]` и `[6, 6]`, их суммы $10 + 7 + 4$ дают 21.

При замене внешнего `A[2]` с 7 на 6 меняю лист с внутренним индексом 10. Далее пересчитываю узлы 5, 2 и 1. Суммы на этом пути становятся 9, 16 и 35. Полная сумма уменьшилась на 1, а диапазоны, не содержащие индекс 2, сохранили результат.

Построение имеет время $O(n)$, потому что $S < 2n$ для $n > 1$, а количество заполненных ячеек пропорционально n . Запрос и точечная замена имеют время $O(\log n)$, память $O(n)$. Корректно обработан и массив, длина которого не является степенью двойки: пустые листья не влияют на ответ.

Задание 12 Сумма, минимум и максимум

Условие задания

1. Расширьте дерево отрезков так, чтобы каждый узел хранил сумму, минимальное и максимальное значение своего диапазона.
2. Реализуйте запрос, возвращающий три характеристики заданного диапазона.
3. Проверьте работу на нескольких диапазонах различной длины, включая один элемент и весь массив.
5. Объясните, почему дерево Фенвика удобно для суммы, а дерево отрезков проще расширить для минимума и максимума.

Ход решения

Для каждого узла с индексом i я храню три характеристики в одноимённых позициях трёх массивов: `tree[i]` — сумма, `minimum[i]` — минимум, `maximum[i]` — максимум. Это одна структура дерева с тремя полями узла, а не три разных разбиения массива.

В листе все три характеристики равны его элементу. Родитель складывает суммы, выбирает меньший из двух минимумов и больший из двух максимумов. Эти действия выполняет `pull`. `query_stats` выбирает те же сегменты, что запрос суммы, и собирает сразу три результата.

Пустым дополнительным листьям задаю сумму 0, минимум плюс бесконечность и максимум минус бесконечность. Поэтому они не добавляют сумму и не могут стать настоящим минимумом или максимумом. Это важно, например, для массива из семи отрицательных чисел.

Общий класс `SegmentTree` полностью приведён в задании 11. Здесь я повторно использую его без копирования. В новом примере сравниваю `query_stats` со значениями `sum()`, `min()` и `max()` на каждом диапазоне и проверяю отрицательное обновление.

Код `main.py task12`

```
# Объявляю демонстрацию задания 12; она выполняется при вызове.
def task12():
```

```

# Создаю независимую копию примера, чтобы обновления не влияли на другие задания.
data = EXAMPLE.copy()
# Строю дерево отрезков для текущего массива.
tree = SegmentTree(data)
# Проверяю шесть заранее заданных диапазонов разной длины.
for left, right in CHECK_RANGES:
    # Беру срез для проверки; right + 1 включает правый элемент.
    part = data[left:right + 1]
    # Получаю эталонную тройку обычными функциями Python.
    expected = (sum(part), min(part), max(part))
    # Вывожу «(Сумма, минимум, максимум): Обычные функции:» и соответствующие
    # значения.
    print((left, right), "(Сумма, минимум, максимум):", tree.query_stats(left, right),
          # Продолжение print: вывожу эталонную тройку sum, min и max.
          "Обычные функции:", expected)
# Меняю шестое значение на отрицательное, чтобы проверить минимум.
data[5] = -5
# Обновляю лист и его предков в дереве отрезков.
tree.update(5, -5)
# Вывожу «После A[5] = -5, весь массив:» и соответствующие значения.
print("После A[5] = -5, весь массив:", tree.query_stats(0, 7))

```

Результат выполнения

```

(0, 0) (Сумма, минимум, максимум): (5, 5, 5) Обычные функции: (5, 5, 5)
(0, 7) (Сумма, минимум, максимум): (36, 1, 8) Обычные функции: (36, 1, 8)
(2, 6) (Сумма, минимум, максимум): (21, 1, 7) Обычные функции: (21, 1, 7)
(1, 5) (Сумма, минимум, максимум): (19, 1, 7) Обычные функции: (19, 1, 7)
(7, 7) (Сумма, минимум, максимум): (8, 8, 8) Обычные функции: (8, 8, 8)
(3, 4) (Сумма, минимум, максимум): (9, 3, 6) Обычные функции: (9, 3, 6)
После A[5] = -5, весь массив: (30, -5, 8)

```

Объяснение результата и вывод

Для $[0, 7]$ получена тройка $(36, 1, 8)$, для одного элемента $[7, 7]$ — $(8, 8, 8)$. После замены $A[5]$ с 1 на -5 весь массив имеет характеристики $(30, -5, 8)$. Сумма уменьшилась на 6, минимум изменился, максимум остался 8.

Обычный Фенвик удобен для суммы: из суммы большого префикса можно вычесть меньший. Минимум или максимум так вычитать нельзя. Например, если минимум большого префикса равен 1, а меньшего тоже 1, разность 0 ничего не говорит о минимуме оставшегося диапазона.

Вывод: дерево отрезков проще расширить для нескольких объединяемых характеристик. Запрос сразу трёх величин всё равно имеет время $O(\log n)$, потому что число характеристик постоянно.

Задание 13 Самостоятельная работа

Условие задания

Разработайте программу для анализа последовательности измерений датчика. Имеется набор датчиков, каждый из которых определяется уникальным целочисленным идентификатором, а для выбранного датчика хранится последовательность числовых измерений.

Программа должна:

- поддерживать поиск датчика по идентификатору с использованием бинарного дерева поиска;
- выводить идентификаторы датчиков в отсортированном порядке с помощью обхода дерева;
- хранить не менее 1000 измерений выбранного датчика;
- вычислять сумму значений на заданном диапазоне не менее чем двумя способами;
- вычислять минимум и максимум на диапазоне с использованием дерева отрезков;
- позволять изменить отдельное измерение и после изменения получать корректные результаты запросов;
- сравнить выбранные структуры по поддерживаемым операциям и их вычислительной сложности.

Структуру программы, набор функций и классов, а также конкретные тестовые данные необходимо определить самостоятельно. В отчёте необходимо показать примеры поиска, диапазонных запросов до и после обновления и кратко обосновать выбор каждой структуры данных.

Ход решения

Я создаю BST из семи идентификаторов датчиков. Проверяю поиск существующего 8 и отсутствующего 999. Отсортированный список получаю inorder-обходом, а не готовой сортировкой. Для анализа выбираю датчик 8 и сначала убеждаюсь через search, что он найден.

Для выбранного датчика создаю 1000 учебных измерений. Значение $i \% 100$ повторяет числа от 0 до 99 десять раз. Это позволяет легко проверить общую сумму: одна сотня даёт 4950, десять сотен — 49500. В этой демонстрации последовательность хранится только для выбранного датчика; остальные идентификаторы нужны для проверки поиска и порядка.

Сумму одного и того же диапазона получаю тремя способами: обычным циклом, Фенвиком и деревом отрезков. Минимум и максимум беру из query_stats. Затем меняю измерение 125 с 25 на 200, обновляю исходный список и обе структуры и повторяю те же запросы.

Для Фенвика передаю $\text{delta} = 175$, для дерева отрезков $\text{value} = 200$. Среди запросов есть один изменённый элемент, весь массив, диапазон с индексом 125 и два диапазона без него. Так проверяю не только изменение нужных результатов, но и сохранение остальных.

Код main.py task13

```
# Объявляю демонстрацию задания 13; она выполняется при вызове.
def task13():
    # Строю новое BST из семи ключей; предыдущие задания его не изменяют.
    root = build_bst(KEYS)
    # Вывожу «Идентификаторы датчиков по порядку:» и соответствующие значения.
    print("Идентификаторы датчиков по порядку:", inorder(root))
    # Проверяю существующий датчик 8 и отсутствующий датчик 999.
    for sensor_id in [8, 999]:
        # Ищу идентификатор именно в BST и получаю число посещённых узлов.
        node, visited = search(root, sensor_id)
        # Вывожу «Датчик: Найден: Посещено:» и соответствующие значения.
        print("Датчик:", sensor_id, "Найден:", node is not None, "Посещено:", visited)
    # Выбираю датчик 8 для анализа последовательности измерений.
    selected_id = 8
    # Проверяю наличие выбранного датчика поиском в дереве.
    node, visited = search(root, selected_id)
    # Если датчик отсутствует, анализ его измерений продолжать нельзя.
    if node is None:
        # Вывожу «Выбранный датчик не найден» и соответствующие значения.
        print("Выбранный датчик не найден")
        # Немедленно завершаю функцию; ниже в этой ветви делать нечего.
        return
    # Создаю пустой список исходных значений.
    data = []
    # Выполняю 1000 шагов для создания запросов или измерений.
    for i in range(1000):
```

```

# Добавляю остаток от деления i на 100; значения 0..99 повторяются десять раз.
data.append(i % 100)
# Строю Фенвик для сумм и точечных прибавлений к измерениям.
fenwick = FenwickTree(data)
# Строю дерево отрезков для суммы, минимума и максимума измерений.
segment = SegmentTree(data)
# Выбираю затронутые и незатронутые диапазоны, один элемент и весь массив.
queries = [(0, 9), (120, 130), (125, 125), (0, 999), (700, 709)]
# Вывожу «Выбран датчик: Количество измерений:» и соответствующие значения.
print("Выбран датчик:", selected_id, "Количество измерений:", len(data))
# Выполняю одинаковые запросы до обновления и после него.
for stage in ["До изменения", "После изменения"]:
    # Во втором этапе сначала изменяю измерение, затем повторяю запросы.
    if stage == "После изменения":
        # Выбираю измерение с индексом 125; его старое значение равно 25.
        index = 125
        # Задаю новое значение выбранного измерения, равное 200.
        value = 200
        # Для Фенвика вычисляю прибавку 200 - 25 = 175.
        delta = value - data[index]
        # Обновляю исходный список, чтобы обычный цикл проверял новые данные.
        data[index] = value
        # Прибавляю delta к нужным блокам Фенвика.
        fenwick.update(index, delta)
        # Заменяю лист дерева отрезков новым значением value.
        segment.update(index, value)
        # Вывожу «Измерение 125: 25 -> 200; delta =» и соответствующие значения.
        print("Измерение 125: 25 -> 200; delta =", delta)
    # Вывожу полученный результат текущего этапа.
    print(stage)
    # Перебираю заранее подготовленные пары границ запросов.
    for left, right in queries:
        # Получаю сумму обычным циклом как простой эталон.
        naive = range_sum_naive(data, left, right)
        # Получаю сумму того же диапазона через Фенвик.
        fast = fenwick.range_sum(left, right)
        # Распаковываю тройку дерева отрезков в сумму, минимум и максимум.
        total, smallest, largest = segment.query_stats(left, right)
        # Вывожу «Цикл: Фенвик: Дерево: Минимум: Максимум: Суммы совпали:» и
        # соответствующие значения.
        print((left, right), "Цикл:", naive, "Фенвик:", fast,
              # Продолжение print: вывожу сумму, минимум и максимум дерева отрезков.
              "Дерево:", total, "Минимум:", smallest, "Максимум:", largest,
              # Сравниваю все три суммы; True означает, что они одинаковы.
              "Суммы совпали:", naive == fast == total)

```

Результат выполнения

Идентификаторы датчиков по порядку: [2, 4, 6, 8, 10, 12, 14]
 Датчик: 8 Найден: True Посещено: 1
 Датчик: 999 Найден: False Посещено: 3
 Выбран датчик: 8 Количество измерений: 1000

Изменение: измерение с индексом 125 заменено с 25 на 200, delta = 175. Ниже приведены все запросы из настоящего вывода. Во всех строках три суммы совпали.

Этап	Диапазон	Цикл	Фенвик	Дерево	Мин	Макс
До	[0, 9]	45	45	45	0	9
До	[120, 130]	275	275	275	20	30
До	[125, 125]	25	25	25	25	25
До	[0, 999]	49500	49500	49500	0	99

Этап	Диапазон	Цикл	Фенвик	Дерево	Мин	Макс
До	[700, 709]	45	45	45	0	9
После	[0, 9]	45	45	45	0	9
После	[120, 130]	450	450	450	20	200
После	[125, 125]	200	200	200	200	200
После	[0, 999]	49675	49675	49675	0	200
После	[700, 709]	45	45	45	0	9

Объяснение результата и вывод

Во всех десяти запросах до и после изменения три суммы совпали. Для [120, 130] сумма выросла с 275 до 450, то есть на 175. Новый максимум равен 200. Для [125, 125] сумма, минимум и максимум равны самому новому измерению 200.

Полная сумма изменилась с 49500 до 49675. Запросы [0, 9] и [700, 709] сохранили сумму 45, минимум 0 и максимум 9. Эти диапазоны не содержат индекс 125, поэтому обновление не должно было на них повлиять.

BST выбран для поиска идентификатора и вывода идентификаторов по порядку. Фенвик выбран для компактного хранения изменяемых сумм. Дерево отрезков нужно для суммы, минимума и максимума после замен. Обычный цикл остаётся простым способом проверки. Сравнение сложности приведено после результатов.

Сравнение структур для измерений

Структура	Построение	Сумма	Изменение	Назначение
Обычный список	$O(n)$	$O(n)$ в худшем случае	$O(1)$	Простая проверка
Префиксы	$O(n)$	$O(1)$	$O(n)$	Статические суммы
Фенвик	$O(n \log n)$	$O(\log n)$	$O(\log n)$	Изменяемые суммы
Дерево отрезков	$O(n)$	$O(\log n)$	$O(\log n)$	Сумма, min, max

Для всех перечисленных хранилищ данных требуется $O(n)$ памяти. У обычного цикла дополнительная память запроса $O(1)$; вспомогательные структуры занимают $O(n)$. У Фенвика здесь указано именно построение через повторные update. Минимум и максимум обычным циклом имеют время $O(n)$, через дерево отрезков — $O(\log n)$.

Отдельно для идентификаторов датчиков BST имеет поиск и вставку $O(h)$, в сбалансированном случае $O(\log m)$, в худшем $O(m)$, где m — число датчиков. Inorder перечисляет все идентификаторы за $O(m)$. В моём наборе семь датчиков, дерево имеет высоту 2.

Проверка правильности

Я проверил три точных порядка обхода, отсутствие ключа, повторную вставку, три случая удаления и четыре поворота. Дополнительно проверил, что поворот не теряет непустую промежуточную ветвь. Высота и число посещений совпали с ожидаемыми 2 и 6, 3 и 7.

Для массивов длиной 1, 7, 8 и 17 я сравнил каждый возможный непустой диапазон с `sum()`, `min()` и `max()`. Проверил отрицательные исходные значения, обновление первого и последнего элемента, повторные изменения, ноль и новые отрицательные значения. Это проверяет одиночный массив, границы и длины, не равные степени двойки. Некорректные границы и индексы Фенвик и дерево отрезков отклоняют.

Все 13 заданий завершились без ошибок. В опыте на 100000 элементах совпали все 1000 ответов двух способов подсчёта. В самостоятельной работе совпали все три суммы в каждом из пяти диапазонов до обновления и после него. Запуск отдельного задания и сообщение о неверном номере тоже проверены.

Заключение

Я реализовал узел дерева, основные операции BST, три обхода и четыре вида поворотов. Сравнение деревьев показало, что форма влияет на число посещений: для семи одинаковых ключей поиск 14 потребовал 3 или 7 узлов. Поэтому быстрый поиск связан с высотой, а балансировка нужна для её ограничения.

На одинаковых 100000 числах и 1000 диапазонах префиксные суммы дали те же ответы, что обычный цикл, при меньшем времени запросов. Но замена одного элемента показала необходимость исправлять последующие префиксы. Фенвик и дерево отрезков поддерживают точечные изменения за $O(\log n)$; дерево отрезков дополнительно возвращает минимум и максимум.

В программе датчиков поиск выполняется через BST, а после замены измерения 125 все три способа подсчёта суммы продолжают совпадать. Изменённые диапазоны дали новые результаты, незатронутые сохранили прежние. Я выбираю структуру по нужным операциям: порядок ключей, частота обновлений и вид диапазонного запроса.

Контрольные вопросы

1. Что называется деревом и какие основные элементы используются при его описании?

Дерево — структура из узлов и связей, образующих иерархию. У корня нет родителя; у остальных узлов есть один непосредственный родитель. Потомок расположен ниже родителя и связан с ним ребром. Лист не имеет потомков. Поддерево состоит из выбранного узла и всех его потомков. Глубина — число рёбер от корня до узла, высота дерева — максимальная глубина. В примере работы корень 8, листья 2, 6, 10 и 14, а узел 4 вместе с 2 и 6 образует поддерево.

2. Что такое высота дерева и как она влияет на стоимость операций?

Высота — число рёбер на самом длинном пути от корня до листа. У листа она равна 0. Поиск, вставка и удаление BST проходят по ветвям, поэтому их стоимость $O(h)$. Для сбалансированного дерева h имеет порядок $\log n$, для цепочки $h = n - 1$. В задании 6 высоты 2 и 6 привели к трём и семи посещениям при поиске одного ключа 14. Чем выше дерево при том же числе ключей, тем больше сравнений может понадобиться.

3. Чем бинарное дерево поиска отличается от произвольного бинарного дерева?

У произвольного бинарного дерева каждый узел имеет не больше двух потомков, но значения могут располагаться как угодно. В BST дополнительно действует порядок: все ключи левого поддерева меньше текущего, а правого больше. Именно порядок позволяет по одному сравнению выбрать ветвь поиска. Если порядок отсутствует, нужное значение может быть в любой ветви, и приходится просматривать узлы. Для повторных ключей требуется отдельное правило; в моей реализации повтор не добавляется.

4. Как выполняются поиск и вставка ключа в BST?

Поиск начинается с корня. Если ключ совпал, возвращается узел. Если он меньше, переход выполняется влево, если больше — вправо. Пустая ссылка означает, что ключ отсутствует. Вставка использует те же сравнения, но в первой нужной пустой позиции создаёт новый узел. При рекурсивной вставке возвращённый корень ветви записывается обратно в left или right. Обе операции зависят от высоты дерева и имеют время $O(h)$.

5. Какие случаи необходимо учитывать при удалении узла из BST?

Нужно учитывать лист, один потомок и два потомка. Лист заменяется пустой ссылкой. Узел с одним потомком заменяется этим потомком вместе с его поддеревом. При двух потомках удобно найти минимальный узел правой ветви, перенести его ключ и удалить прежнее вхождение справа. В работе после удаления 2 узел 4 имел одного потомка 6; при удалении 8 с двумя потомками использован ключ 10. Во всех случаях порядок BST должен сохраниться.

6. Чем отличаются preorder, inorder и postorder?

Preorder посещает текущий узел, затем левую и правую ветви. Inorder посещает левую ветвь, текущий узел и правую ветвь. Postorder посещает обе ветви, затем текущий узел. Поэтому preorder начинает вывод с корня, postorder заканчивает корнем, а inorder выводит ключи BST по возрастанию. Эти обходы применяются к разным задачам: например, postorder удобен, когда сначала нужны результаты потомков при вычислении дерева выражения.

7. Почему inorder-обход BST возвращает ключи в отсортированном порядке?

В левом поддереве BST находятся меньшие ключи, чем в текущем узле. Inorder сначала выводит их, затем текущий ключ, затем большие ключи правой ветви. То же правило повторяется рекурсивно внутри каждой ветви. В результате меньшие значения всегда появляются раньше больших. В моём дереве результат [2, 4, 6, 8, 10, 12, 14]. Это свойство BST, а не любого бинарного дерева; при произвольном расположении значений inorder не будет сортировать их.

8. Почему BST может вырождаться при неудачном порядке вставки?

Форма обычного BST определяется порядком вставки. Если вставлять возрастающие ключи, каждый новый ключ идёт вправо от всех предыдущих. Вместо дерева с короткими ветвями получается цепочка. У неё n узлов и $n - 1$ рёбер на самом длинном пути, поиск последнего ключа просматривает все n узлов. В задании 6 последовательность 2, 4, 6, 8, 10, 12, 14 дала высоту 6. Чтобы высота не зависела так сильно от порядка поступления, используются самобалансирующиеся деревья.

9. Что такое баланс-фактор AVL-узла?

Баланс-фактор AVL-узла равен высоте его левого поддерева минус высота правого. Значение 0 означает равные высоты, 1 — левая ветвь выше на одно ребро, -1 — правая выше на одно ребро. Для AVL допустимы только -1, 0 и 1 у каждого узла. Значения 2 и -2 указывают на нарушение баланса. После поворотов примеров задания 7 баланс-фактор корня 20 равен 0.

10. Для чего используются одинарные и двойные повороты AVL-дерева?

Повороты меняют локальные связи, уменьшают перекося и сохраняют порядок BST. Для LL нужен правый поворот, для RR — левый. В зигзагах LR и RL одного такого поворота недостаточно: сначала поворачивается дочерняя ветвь, затем проблемный корень. LR исправляется левым и правым поворотами, RL — правым и левым. Промежуточное поддерево нельзя терять; оно переподключается к прежнему корню. Набор ключей и inorder остаются прежними.

11. Чем в общем виде красно-чёрное дерево отличается от AVL-дерева?

AVL напрямую ограничивает разницу высот левой и правой ветвей у каждого узла. Красно-чёрное дерево задаёт ограничения через цвета: корень чёрный, у красного узла не бывает красного потомка, на путях к пустым листьям сохраняется одинаковое число чёрных узлов. Баланс у него менее строгий, но высота тоже остаётся $O(\log n)$. Поэтому оба вида дают логарифмические поиск, вставку и удаление, используя разные правила восстановления структуры.

12. Почему B-Tree и B+Tree удобны для внешней памяти?

При хранении на диске важно число обращений к блокам внешней памяти. У бинарного дерева в узле один ключ, поэтому много переходов может означать много чтений. B-Tree и B+Tree хранят несколько ключей в одном узле и имеют много потомков. Они получаются широкими и низкими, а размер узла можно согласовать с размером дискового блока. Один прочитанный узел сразу помогает выбрать направление среди нескольких диапазонов. Поэтому такие деревья удобны для индексов и больших хранилищ.

13. В чём основная особенность организации данных в B+Tree?

В B+Tree внутренние узлы в основном направляют поиск, а сами записи или ссылки на них находятся в листьях. Листья связаны между собой. После нахождения начала диапазона можно последовательно читать соседние листья, не выполняя поиск каждого следующего ключа от корня. Поэтому структура подходит одновременно для точного поиска и диапазонного чтения. В B-Tree записи могут находиться также во внутренних узлах; это отличает её организацию от B+Tree.

14. Что называется диапазонным запросом и зачем нужна предварительная обработка?

Диапазонный запрос вычисляет характеристику части массива, например сумму, минимум или максимум на [left, right]. Простой перебор имеет время, пропорциональное длине диапазона. Когда запросов много, одни элементы просматриваются повторно. Предварительная обработка создаёт дополнительную структуру один раз и сохраняет нужные промежуточные результаты. Она требует времени и памяти, но позволяет быстрее отвечать на следующие запросы. Выбор структуры зависит также от наличия обновлений.

15. Как строятся префиксные суммы и как с их помощью получить сумму диапазона?

Я создаю `prefix` длиной $n + 1$ и задаю `prefix[0] = 0`. Затем `prefix[i + 1] = prefix[i] + data[i]`. Для включительного `[left, right]` использую `prefix[right + 1] - prefix[left]`. Первая величина содержит сумму до правой границы, вторая удаляет лишнее начало до левой границы. Для `[2, 6]` массива из лекции получается $28 - 7 = 21$. Построение требует $O(n)$, запрос готовых префиксов — $O(1)$, память — $O(n)$.

16. Почему префиксные суммы неудобны при частых изменениях массива?

Изменённый элемент входит во все последующие префиксы. Если `data[index]` меняется на `delta`, нужно исправить `prefix[index + 1]`, `prefix[index + 2]` и далее до `prefix[n]`. В худшем случае это $O(n)$ действий. Если обновлений много, преимущество быстрых запросов может перекрываться постоянными пересчётами. В задании 9 после замены 7 на 10 старый запрос дал 21 вместо 24, поэтому прежнюю структуру использовать без обновления нельзя.

17. Для чего используется дерево Фенвика?

Фенвик используется для изменяемых префиксных сумм и сумм диапазонов. Вспомогательный массив хранит суммы блоков, выбранных по индексам. Для одного запроса или точечного прибавления нужно обработать лишь $O(\log n)$ блоков. Это удобно, когда данные меняются, а нужны в основном суммы. Структура занимает $O(n)$ памяти. В моей реализации построение последовательными `update` имеет $O(n \log n)$; быстрый алгоритм линейного построения здесь не используется.

18. Что определяет выражение `i & -i` в дереве Фенвика?

Выражение `i & -i` выделяет младший установленный бит положительного внутреннего индекса. Это `lowbit` — длина блока, сумму которого хранит `tree[i]`. Для `i = 6` получается 2, поэтому блок охватывает внутренние позиции 5 и 6. Для `i = 8` получается 8, блок охватывает позиции 1..8. При запросе `i` уменьшается на `lowbit`, а при обновлении увеличивается. Индексация с единицы нужна, в частности, чтобы не застрять на нулевом шаге `lowbit(0)`.

19. Как получить сумму диапазона с помощью дерева Фенвика?

Сумма включительного диапазона равна $\text{prefix_sum}(\text{right}) - \text{prefix_sum}(\text{left} - 1)$, если методы используют внешние индексы с нуля. Сначала берётся сумма от начала до правой границы, затем вычитаются элементы перед левой. При $\text{left} = 0$ второй префикс пуст и равен 0. Внутри метода внешний индекс переводится в индекс Фенвика через $+1$. Для массива из лекции запрос $[2, 6]$ даёт $28 - 7 = 21$ и имеет время $O(\log n)$.

20. Что хранит узел дерева отрезков?

Узел дерева отрезков соответствует определённому диапазону массива и хранит его агрегированную характеристику. Корень отвечает за весь массив, листья — за отдельные элементы. Для суммы родитель складывает суммы детей. Для минимума берёт $\min$ двух минимумов, для максимума $\max$ двух максимумов. В работе один узел одновременно имеет эти три характеристики в трёх массивах с одинаковыми индексами. Например, узел диапазона $[0, 3]$ хранит сумму 17, минимум 2 и максимум 7.

21. Как выполняются диапазонный запрос и точечное обновление дерева отрезков?

Запрос объединяет информацию от непересекающихся сегментов, которые вместе покрывают нужный диапазон. Узлы вне запроса игнорируются, подходящие целиком используются без спуска к каждому элементу. В моей реализации границы поднимаются от листьев к корню, а крайние неподходящие пары разделяются. Точечное обновление заменяет лист и пересчитывает его предков. У дерева логарифмическое число уровней, поэтому оба действия имеют время $O(\log n)$. Построение занимает $O(n)$.

22. В каких случаях удобнее применять префиксные суммы, дерево Фенвика и дерево отрезков?

Префиксные суммы выбираю для неизменяемого массива с большим числом сумм: после $O(n)$ подготовки запрос имеет $O(1)$. Фенвик подходит для сумм и точечных прибавлений при изменяемых данных: обе операции $O(\log n)$, структура компактная. Дерево отрезков выбираю, когда кроме суммы нужны минимум, максимум или другие объединяемые характеристики и точечные замены. В датчиках я использую Фенвик для сумм, дерево отрезков для трёх характеристик и обычный цикл для проверки. Если запросов мало, простой перебор может быть достаточен.

Использованные материалы

Лабораторная работа №2 «Деревья и быстрые запросы к данным», МТУСИ, Москва 2026.

Лекция 2 «Деревья и быстрый поиск в упорядоченных данных», Борзов Вадим Максимович, 10 сентября 2026.

Лекция 3 «Быстрые запросы к диапазонам данных», Борзов Вадим Максимович.

Предыдущий отчёт «Быстрый доступ к данным» использован как образец оформления.